

Chapter 10: UNORDERED_MAP (Most Used in Interviews)

If Map is:

```
Sorted Key -> Value  
O(log n)
```

Then Unordered Map is:

```
Unsorted Key -> Value  
O(1) Average
```

This is the container you'll use most often in:

- LeetCode
- Coding Interviews
- Competitive Programming
- Frequency Counting
- Hashing Problems

What Problem Does unordered_map Solve?

Suppose:

```
1 2 2 3 1 1
```

Need frequency:

```
1 -> 3  
2 -> 2  
3 -> 1
```

Use:

```
unordered_map<int,int> freq;
```

Fast.

Internal Structure

Uses:

Hash Table

Unlike map:

```
map -> Red Black Tree
unordered_map -> Hash Table
```

Header

```
#include <unordered_map>
```

Declaration

```
unordered_map<int,int> mp;
```

Insert Data

Method 1

```
mp[1]=100;
mp[2]=200;
```

Method 2

```
mp.insert({3,300});
```

Access

```
cout<<mp[1];
```

Output:

```
100
```

Important Behavior

```
cout<<mp[999];
```

Output:

```
0
```

And key is created automatically.

Exactly like map.

Traversing

```
for(auto x:mp)
{
    cout<<x.first<<" ";
    cout<<x.second<<endl;
}
```

Output order:

✗ Not guaranteed

Example:

```
2 200  
1 100  
3 300
```

or anything else.

Find

```
auto it=mp.find(2);
```

Check:

```
if(it!=mp.end())  
{  
    cout<<"Found";  
}
```

Complexity:

```
O(1)
```

Average.

Count

```
mp.count(2);
```

Returns:

```
1
```

or

```
0
```

Erase

By Key

```
mp.erase(2);
```

By Iterator

```
auto it=mp.find(2);  
mp.erase(it);
```

Size

```
mp.size();
```

Empty

```
mp.empty();
```

Clear

```
mp.clear();
```

Swap

```
mp1.swap(mp2);
```

Frequency Counting

The #1 use.

Input:

```
1 2 2 3 1 1
```

Code:

```
unordered_map<int,int> freq;  
  
for(int x:arr)  
{  
    freq[x]++;  
}
```

Result:

```
1 -> 3  
2 -> 2  
3 -> 1
```

Character Frequency

```
unordered_map<char,int> freq;  
  
for(char ch:s)  
{
```

```
    freq[ch]++;  
}
```

Input:

banana

Possible Output:

```
a -> 3  
n -> 2  
b -> 1
```

Order not guaranteed.

Map vs Unordered Map

Feature	Map	Unordered Map
Structure	Tree	Hash Table
Sorted	Yes	No
Insert	$O(\log n)$	$O(1)$
Find	$O(\log n)$	$O(1)$
Erase	$O(\log n)$	$O(1)$
lower_bound	Yes	No
upper_bound	Yes	No

When to Use Which?

Use Map:

```
Need Sorting  
Need lower_bound  
Need upper_bound  
Need ordered traversal
```

Use Unordered Map:

```
Need Fast Frequency Count  
Need Fast Lookup  
Don't Care About Order
```

Very Common Interview Pattern

Two Sum

Input:

```
nums = [2,7,11,15]  
  
target = 9
```

Solution:

```
unordered_map<int,int> mp;  
  
for(int i=0;i<nums.size();i++)  
{  
    int need=target-nums[i];  
  
    if(mp.count(need))  
        return {mp[need],i};  
  
    mp[nums[i]]=i;  
}
```

Complexity:

```
O(n)
```

Grouping

Example:

```
Department -> Employees
```

Use:

```
unordered_map<string,vector<string>>
```

Example:

```
unordered_map<string,vector<string>> dept;  
  
dept["IT"].push_back("Rahul");  
dept["IT"].push_back("Amit");
```

Nested Map

Possible:

```
unordered_map<int,unordered_map<int,int>>
```

Advanced.

Hash Table Concepts

Interview Theory.

Collision

Two keys produce same hash.

Example:

```
Key 10 -> Bucket 1  
Key 20 -> Bucket 1
```

Collision.

STL handles this automatically.

Bucket Functions

Advanced.

bucket_count()

```
cout<<mp.bucket_count();
```

load_factor()

```
cout<<mp.load_factor();
```

Average elements per bucket.

rehash()

```
mp.rehash(100);
```

Creates more buckets.

Rarely used.

Time Complexity

Operation	Average	Worst
insert	$O(1)$	$O(n)$
erase	$O(1)$	$O(n)$
find	$O(1)$	$O(n)$
count	$O(1)$	$O(n)$
[]	$O(1)$	$O(n)$

Interviewers usually mean average case.

Common Interview Uses

Frequency Count

```
unordered_map<int,int>
```

Character Count

```
unordered_map<char,int>
```

Index Mapping

```
value -> index
```

Fast Lookup

```
if(mp.count(x))
```

Caching

```
key -> computed value
```

Practice Questions

Easy

Q1

Count frequency.

Input:

```
1 2 2 3 1
```

Q2

Character frequency.

Input:

```
banana
```

Q3

Check if key exists.

Q4

Delete key.

Q5

Find most frequent element.

Medium

Q6

First non-repeating character.

Q7

Two Sum.

Q8

Count pairs with given sum.

Q9

Find duplicate elements.

Q10

Group words by first character.

Interview Level

Q11

Two Sum

LeetCode #1

Q12

Subarray Sum Equals K

LeetCode #560

Q13

Longest Consecutive Sequence

LeetCode #128

Q14

Top K Frequent Elements

LeetCode #347

Q15

Group Anagrams

LeetCode #49

Common Mistakes

Mistake 1

Expecting sorted order.

```
unordered_map
```

Never sorted.

Mistake 2

Using:

```
lower_bound()  
upper_bound()
```

✗ Not available.

Mistake 3

Using:

```
mp[key]
```

for checking existence.

This inserts key.

Use:

```
mp.find(key)
```

or

```
mp.count(key)
```

Revision Sheet

Header

```
#include <unordered_map>
```

Declaration

```
unordered_map<Key,Value> mp;
```

Insert

```
mp[key]=value;  
mp.insert({key,value});
```

Access

```
mp[key];
```

Search

```
mp.find(key);  
mp.count(key);
```

Remove

```
mp.erase(key);
```

Utilities

```
mp.size();  
mp.empty();  
mp.clear();  
mp.swap();
```

Hash Functions

```
bucket_count();  
load_factor();
```

```
rehash();
```

Complexity

```
insert O(1)
find   O(1)
erase  O(1)
count  O(1)
```

Average case.

Golden Rule

Need Order?

```
map
```

Need Speed?

```
unordered_map
```

Mental Trigger

When you see:

- Frequency Count
- Two Sum
- Hashing
- Lookup Table
- Count Occurrences
- Fast Search

Think:

```
unordered_map
```

STL Progress So Far

✓ Pair ✓ Vector ✓ Iterators ✓ Algorithms ✓ Stack ✓ Queue ✓ Deque ✓ Set ✓ Unordered Set ✓
Map ✓ Unordered Map

You now know about **80% of STL used in interviews.**
